Université Blida 1
Faculté des Sciences
Département d'Informatique

GL (Génie Logiciel) & RSD (Réseaux et Systèmes Distribués)

Semestre 1

Algorithmique Avancée & Complexité

Mme AROUSSI (s_aroussi@esi.dz)

2026 - 2027

Disponible sur https://sites.google.com/a/esi.dz/informatiqueblida/

PRÉAMBULE

Master	Génie Logiciel (GL)	Réseaux et Systèmes Distribués (RSD)
Prérequis	L'étudiant doit connaître les bases de programmation et des algorithmes itératifs et récursifs ainsi que les structures de données fondamentales : tableaux, fichiers, piles, files, listes et arbres. (Algorithmique 1 et 2)	
Volume horaire hebdomadaire	1.5H Cours + 1.5H TD + 1.5H TP	
Évaluation	Continu + Examen	
Note TD	Interrogation, devoirs, préparation, assiduité, présence, …	
Note TP	*Test TP,* Démonstration	
Coefficient	3	3
Crédit	5	5

OBJECTIFS DU COURS

- Ce module permet d'acquérir les notions nécessaires pour:
  - Construire la ou les solutions à un problème
  - Evaluer les différentes solutions en termes de calcul de complexité
  - Choisir la meilleure solution
  - Analyser et classer les problèmes de différents domaines
- Ces notions seront vues à travers l'étude de problèmes pris dans différents domaines de l'informatique tels que : Les réseaux, les bases de données, …etc.

Contenu du cours

I. Complexité

II. Arbres de Recherche

III. Méthodes Algorithmiques Avancées

IV. NP-Complétude

Université Blida 1
Faculté des Sciences
Département d'Informatique

GL (Génie Logiciel) & RSD (Réseaux et Systèmes Distribués)

Semestre 1

CHAPITRE I:

COMPLEXITÉ ALGORITHMIQUE

Mme AROUSSI (s_aroussi@esi.dz)

2026-2027

Disponible sur
https://sites.google.com/a/esi.dz/informatiqueblida/

Plan de Chapitre II:

- Introduction
- Définitions
- Type de la Complexité
- Notation de Landau
- Classes de complexité
- Calcul de Complexité des Algorithmes Itératifs

INTRODUCTION

- Le temps d'exécution d'un algorithme dépend des facteurs suivants :
  - Les données du programme,
  - La qualité du compilateur (langage utilisé),
  - La machine utilisée (vitesse, mémoire,),
  - La complexité de l'algorithme lui-même,
- On cherche à mesurer la complexité d'un algorithme indépendamment de la machine et du langage utilisés, c.-à-d. uniquement en fonction de la taille des données que l'algorithme doit traiter.

INTRODUCTION

EXEMPLE: CALCUL DE LA VALEUR D'UN POLYNÔME

- Soit P(X) un polynôme de degré n

$P(X) = a_nX^n + a_{n-1}X^{n-1} + ... + a_1X + a_0$,

Où: n : entier naturel

a_n, a_{n-1}, ..., a_1, a_0 : les coefficients du polynôme qui sont stockés dans le tableau T[n+1] d'entiers.

- **Ecrire un algorithme qui permet de calculer la valeur d'un polynôme en un point x.**

INTRODUCTION

EXEMPLE: CALCUL DE LA VALEUR D'UN POLYNÔME

1ère variante

```
Début
  P←T[0]
  Pour i ← 1 à n faire
       P ← P+ T[i] * Puiss (X, i)
Fin
```

1ère Complexité :

n additions

n multiplications

n *puissances*

Au moins 4 variables

2ème variante

```
Début
Inter ← X
P ← T[0]
Pour i← 1 à n faire
       P ← P+ Inter *T[i]
       Inter ← Inter * X
Fin
```

2ème Complexité :

n additions

2*n multiplications

4 variables

INTRODUCTION

EXEMPLE: CALCUL DE LA VALEUR D'UN POLYNÔME

- **3ème variante: Schéma de Horner**

$$P(X) = a_nX^n + a_{n-1}X^{n-1} + \dots + a_2X^2 + a_1X + a_0$$
$$= (a_nX^{n-1} + a_{n-1}X^{n-2} + \dots + a_2X + a_1)X + a_0$$
$$= ((a_nX^{n-1} + a_{n-1}X^{n-2} + \dots + a_2)X + a_1)X + a_0$$
$$= \dots\dots\dots\dots$$
$$= (\dots(((a_nX + a_{n-1})X + a_{n-2})X\dots\dots)X + \dots + a_2)X + a_1)X + a_0$$

3ème variante

```
Début
  P ← T[n]
  Pour i ← n-1 à 0 faire
      P ← P*X + T[i]
Fin
```

3ème Complexité :

n additions

n multiplications

3 variables

INTRODUCTION

EXEMPLE: CALCUL DE LA VALEUR D'UN POLYNÔME

Variantes	Première	Deuxième	Troisième
Complexité en temps (en terme de nombre d'opérations)	n additions n multiplications n *puissances*	n additions 2*n multiplications	n additions n multiplications
Complexité en espace mémoire (Variables)	P,X, i et les variables de la fonction puissance appelée n fois	P,X, i et Inter	P, X, i

- **Nécessité d'estimer la complexité en temps et en espace d'un algorithme avant de l'écrire et l'implémenter**

DÉFINITION

- **La complexité (temporelle) d'un algorithme** est la mesure du nombre d'opérations fondamentales (affectations, comparaisons, arithmétiques, logiques, ….) qu'il effectue sur un jeu de données. Elle est exprimée comme une fonction de la taille du jeu de données.
  - Elle permet en particulier de comparer deux algorithmes traitant le même calcul. En d'autres termes, elle permet de déterminer si un algorithme A est meilleur qu'un algorithme B indépendamment de la machine, du langage de programmation, du compilateur et des détails d'implémentation.

TYPE DE LA COMPLEXITÉ

Complexité au meilleur

- C'est le plus petit nombre d'opérations qu'aura à exécuter l'algorithme sur un jeu de données de taille n.
- $T_{min}(n) = min_{d \in D_n} T(d)$

Complexité en moyenne

- C'est la moyenne des complexités de l'algorithme sur des jeux de données de taille n
- $T_{moy}(n) = \Sigma_{d \in D_n} T(d) / |D_n|$

Complexité au pire

- C'est le plus grand nombre d'opérations qu'aura à exécuter l'algorithme sur un jeu de données de taille n
- $T_{max}(n) = max_{d \in D_n} T(d)$

- **Notations:**
  - D_n l'ensemble des données de taille **n**
  - **T(n)** le nombre d'opération sur un jeu donnée de taille **n**

NOTATION DE LANDAU

- **La notation de Landau « O »** est celle qui est le plus communément utilisée pour expliquer formellement les performances d'un algorithme.

- Cette notation exprime **la limite supérieure** d'une fonction dans un facteur constant.

$$\mathbf{f = O(g) \Leftrightarrow \exists n_0 , \exists c \geq 0, \forall n \geq n_0 , f(n) \leq c * g(n)}$$

- **Exemple:** $T(n) = O(n^2)$ veut dire qu'il existe une constante $c > 0$ et une constante $n_0 > 0$ tel que pour tout $n > n_0$ $T(n) <= c\ n^2$

NOTATION DE LANDAU

- Les règles de la notation O sont les suivantes :
  - Les termes constants : **O(c) = O(1)**
  - Les constantes multiplicatives sont omises :

$$O(cT) = c\,O(T) = O(T)$$

  - L'addition est réalisée en prenant le maximum :

$$O(T1) + O(T2) = O(T1 + T2) = \max(O(T1);O(T2))$$

  - La multiplication reste inchangée

$$O(T1)O(T2) = O(T1T2)$$

NOTATION DE LANDAU

- Supposant que le temps d'exécution d'un algorithme est décrit par la fonction T(n) = $3n^2+10n+10$, **Calculer O(T(n))?**

$O(T(n)) = O(3\ n^2 + 10n + 10)$

$= O(\max(3\ n^2, 10n, 10))$

$= O(3\ n^2)$

$= O\ (n^2)$

- **Remarque:**

Pour n = 10 nous avons :

  - Temps d'exécution de $3\ n^2$: 3(10)2 / 3(10)2+10(10)+10 = 73,2%
  - Temps d'exécution de 10n : 10(10) / 3(10)2+10(10)+10 = 24,4%
  - Temps d'exécution de 10 : 10 / 3(10)2+10(10)+10 = 2,4%

Le poids de $3\ n^2$ devient encore plus grand quand n = 100, soit 96,7%

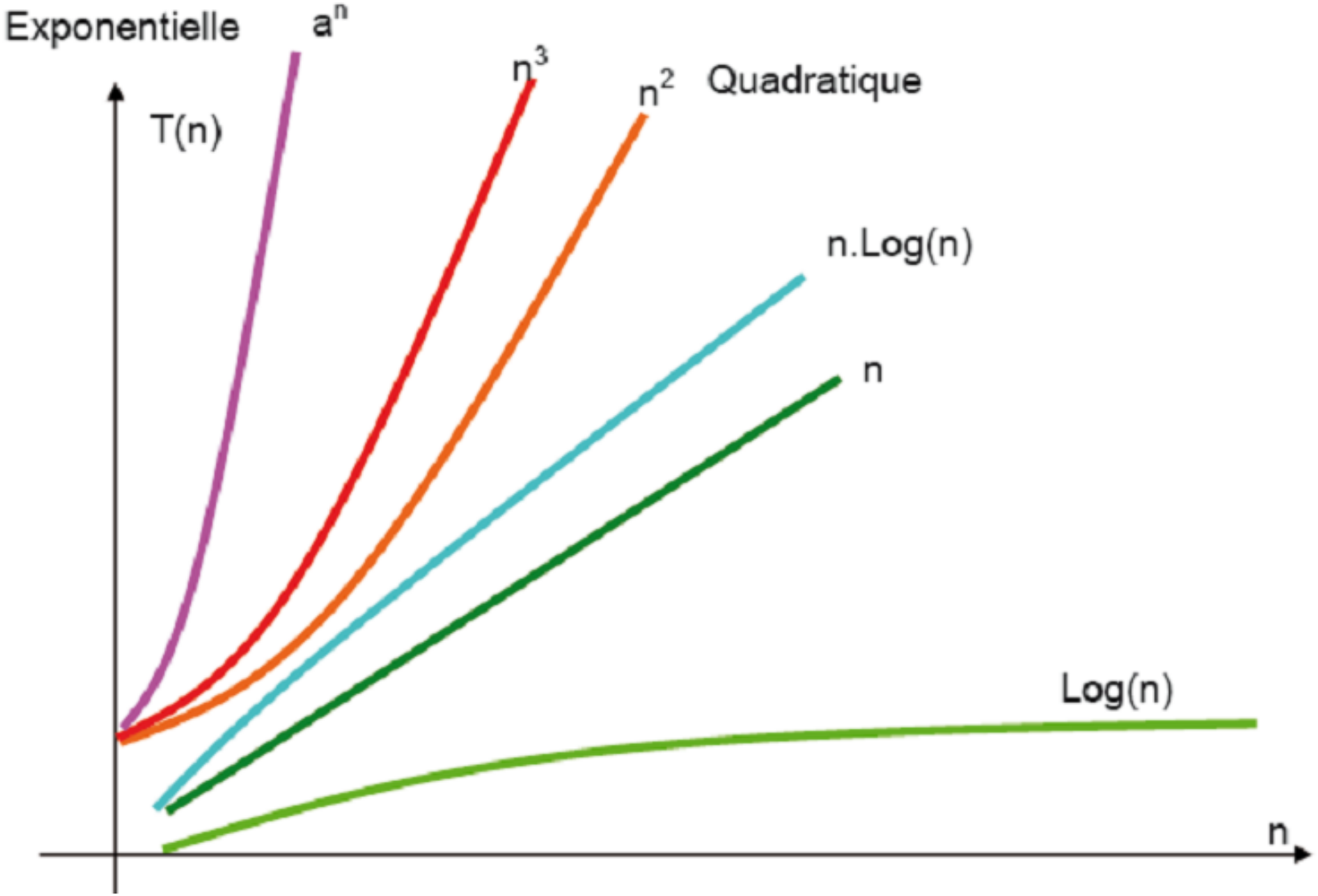
On peut négliger les quantités 10n et 10.

Ceci explique les règles de la notation O.

Classes de complexité

Classe	Notation O	Exemple
Constante	$O(1)$	Accéder au premier élément d'un ensemble de données
Linéaire	$O(n)$	Parcourir un ensemble de données
Logarithmique	$O(\log(n))$	Couper un ensemble de données en deux parties égales, puis couper ces moitiés en deux parties égales, etc.
Quasi-linéaire	$O(n \log(n))$	Couper répétitivement un ensemble de données en deux et combiner les solutions partielles pour calculer la solution générale
Quadratique	$O(n^2)$	Parcourir un ensemble de données en utilisant deux boucles imbriquées
Polynomiale	$O(n^P)$	Parcourir un ensemble de données en utilisant P boucles imbriquées
Exponentielle	$O(a^n)$	Générer tous les sous-ensembles possibles d'un ensemble de données

Classes de complexité



CLASSES DE COMPLEXITÉ

- **Application Numérique:**
  - Dans le tableau (a), il s'agit de calculer le nombre d'opérations nécessaire pour exécuter l'algorithme en fonction de sa complexité et de la taille d'instance du problème traité.

(a) Nombre d'opérations.

Complexité	n=10	n=100	n=1000
n	10^1	10^2	10^3
n^2	10^2	10^4	10^6
n^3	10^3	10^6	10^9
2^n	10^3	10^{30}	10^{300}
$\sqrt{n}$	3.3	10	31
$\sqrt[3]{n}$	2.15	4.64	10
$\log_2(n)$	3.3	6.6	9.9

CLASSES DE COMPLEXITÉ

- **Application Numérique :**
  - Dans le tableau (b), il s'agit de calculer le temps nécessaire à cela en supposant que l'on dispose d'un ordinateur capable de réaliser 10^9 opérations par seconde.

note 1: 30000 milliards d'années, 2300 fois l'âge de l'univers (estimé à 13,7 milliards d'années)

(b) Temps nécessaire à 10^9 op/sec.

Complexité	n=10	n=100	n=1000
n	10ns	100ns	$1\mu s$
n^2	100ns	$10\mu s$	1ms
n^3	$1\mu s$	1ms	1s
2^n	$>1\mu s$	(note 1)	10^{284} ans
$\sqrt{n}$	3.3ns	10ns	31ns
$\sqrt[3]{n}$	2.15ns	4.64ns	10ns
$\log n$	3.3ns	6.6ns	9.9ns

CLASSES DE COMPLEXITÉ

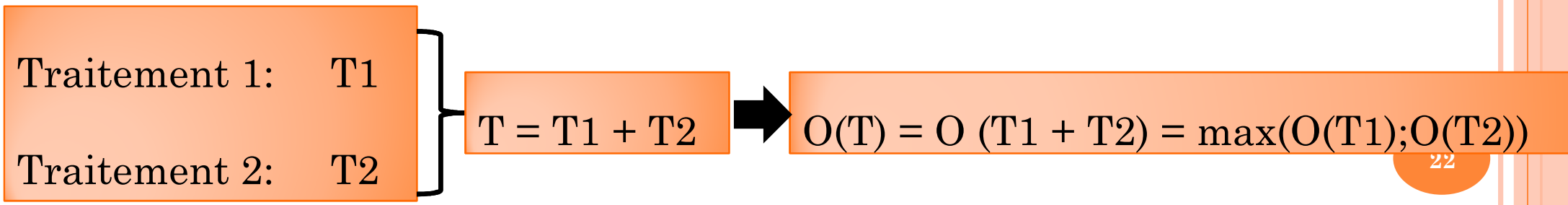
- **Application Numérique:**
  - Dans le tableau (c), on calcule la plus grande instance de problème traitable dans le temps imparti.

(c) Plus grande instance faisable à 10^9 op/sec.

Complexité	1s	1h	1 an
n	10^9	10^{12}	10^{16}
n^2	$3 \cdot 10^4$	10^6	10^8
n^3	10^3	10^4	$2 \cdot 10^5$
2^n	29	39	53
$\sqrt{n}$	10^{18}	10^{24}	10^{32}
$\sqrt[3]{n}$	10^{27}	10^{36}	10^{48}
$\log(n)$	$10^{30\,000\,000}$	∞	∞

CALCUL DE LA COMPLEXITÉ

1. **Cas d'une instruction simple (écriture, lecture, affectation, comparaison, arithmétique, logique) :** Le temps d'exécution de chaque instruction simple est O(1).
2. **Cas d'une suite d'instructions**: Le temps d'exécution d'une séquence d'instruction est déterminée par la règle de la somme. C'est donc le temps de la séquence qui a le plus grand temps d 'exécution:



Calcul de la Complexité

- **Exemple 1: Permutation (Var a, b: entier)**

Instruction	**Temps d'exécution**
x←a	**C1**
a←b	**C2**
b←x	**C3**

$$T = C1 + C2 + C3 = C \Rightarrow O(T) = O\,(C1 + C2 + C3) = O(C) = O(1)$$

Calcul de la Complexité

3. **Cas d'un traitement conditionnel:** Le temps d'exécution d'une instruction SI est le temps d ’exécution des instructions exécutées sous condition, plus le temps pour évaluer la condition. Pour une alternative, on se place dans le cas le plus défavorable.

Si (condition) **alors**

Traitement 1

Sinon

Traitement 2



$T = T_{condition} + \max(T1, T2)$

$O(T) = O(T_{condition}) + \max(O(T1), O(T2))$

$O(T) = \max(O(T_{condition}), O(T1), O(T2))$

Calcul de la Complexité

- **Exemple 2: Minimum entre deux entiers (Min (a,b))**

Instruction	**Temps d'exécution**
Si (a < b)	**C1 ($T_{condition}$)**
Retourner (a)	**C2**
Sinon	
Retourner (b)	**C3**

$$T = C1 + \max(C2, C3) \Rightarrow$$

$$O(T) = O(C1 + \max(C2,C3)) = O(C1+C) = O(C') = O(1)$$

Calcul de la Complexité

4. **Cas d'un traitement répétitif (boucles) :** Le temps d'exécution d'une boucle est la somme du temps pour évaluer le corps et du temps pour évaluer la condition. Souvent ce temps est le produit du nombre d'itérations de la boucle par le plus grand temps possible pour une exécution du corps.

Boucle	Pour	Tant que	Répéter
Forme	Pour i←**id** à **if** pas **b** faire Traitement_corps	Tant que (non condstop) faire Traitement_corps	Répéter Traitement_corps Jusqu'à (condstop)
Temps d'exécution	T = Nombre_itération * (Tcond+Tcorps)		
O-Notation	O(T) = O(Nombre_itération * max (O(Tcond), O(Tcorps))		

Calcul de la Complexité

- **Exemple 3: Minimum d'un tableau entier de taille n (MinTab(T,n))**

Instruction	**Temps d'exécution**
m←T[0]	**C1**
Pour i allant de 1 à n-1 faire	**Condition: (i++; i<n) → C2** **Nombre d'itération: n-1**
Si m>T[i] alors	**C3**
m←T[i]	**C4**
Retourner (m)	**C5**

Remarque:

En général, le nombre d'itératio

de la boucle suivante:

Pour i← « id » à « if » pas « a »

est égale à **|(if-id)/a |+ 1**

$$T = C1 + (n-1) * (C2 + C3 + C4) + C5$$

$$= (C1 + C5 - (C2 + C3 + C4)) + n\,(C2 + C3 + C4) = C6 + n * C7 \Rightarrow$$

$$O(T) = \max(O(C6), O(n * C7)) = \max(O(1), O(n)) = O(n) \text{ linéaire}$$

CALCUL DE LA COMPLEXITÉ

- **Exemple 4: Recherche séquentielle d'un élément x dans un tableau ordonné de taille n (RechSeq(T,x,n))**

Instruction	**Temps d'exécution**
existe←0; i←0;	**C1**
TQ ((i<n) et (non existe) et (x < T[i])) faire	**Condition:C2** **Nombre d'itération: n**
Si T[i] = x alors	**C3**
Existe ←1	**C4**
Sinon i++	**C5**
Retourner (existe)	**C6**

Remarque:

Au pire de cas, parcourir tous le tableau **(i allant de 0 à n-1)** pour conclure que x n'existe pas.

T = C1 + n (C2+ C3+max (C4, C5)) + C6 = C7 + n C8 ⇒

O(T) = max (O(C7),O(nC8)) = max (O(1), O(n)) = O(n) linéaire

CALCUL DE LA COMPLEXITÉ

- **Exemple 5: recherche dichotomique dans un tableau de taille n**



Instructions	**Temps d'exécution**
Binf←0 ; Bsup←n-1	**C1**
Tant que (Binf≤Bsup) faire	**Condition:C2** **Nombre d'itération: ???**
mil ← (Binf+Bsup) DIV 2 ;	**C3**
Si (T[mil]=x) Alors retourner (1);	**C4**
Sinon Si (x < T[mil]) Alors Bsup ← mil-1 ;	**C5**
Sinon Binf ← mil+1 ;	**C6**
Retourner (0) ;	**C7**

T = (C1 + C7) + q * (C2 + C3 + max (C4, C5, C6)) = C8 + q * C9

Calcul de la Complexité

- **Exemple 5: recherche dichotomique**

Instructions
Binf←0 ; Bsup←n-1
Tant que (Binf≤Bsup) faire
mil ← (Binf+Bsup) DIV 2 ;
Si (T[mil]=x) Alors retourner (1);
Sinon Si (x < T[mil]) Alors Bsup ← mil-1 ;
Sinon Binf ← mil+1 ;
Retourner (0) ;

Remarque:

La taille du tableau : n = Bsup-Binf+1

Cette taille est divisée en 2 à chaque itération

Au pire de cas, on arrive à un tableau à une case **(n, n/2, n/4, n/8, ……., 1)** pour conclure que x n'existe pas.

Nombre d'itérations	**La taille de tableau n**
1	**n**
2	**n/2**
3	**$n/2^2$**
4	**$n/2^3$**
…..	
q	**$n/2^{q-1}= 1$**

CALCUL DE LA COMPLEXITÉ

- **Exemple 5: recherche dichotomique**

Instructions
Binf←0 ; Bsup←n-1
Tant que (Binf≤Bsup) faire
mil ← (Binf+Bsup) DIV 2 ;
Si (T[mil]=x) Alors retourner (1);
Sinon Si (x < T[mil]) Alors Bsup ← mil-1 ;
Sinon Binf ← mil+1 ;
Retourner (0) ;

Remarque: La taille du tableau : n = Bsup-Binf+1

Cette taille est divisé en 2 à chaque itération

Au pire de cas, on arrive à un tableau à une case **(n, n/2, n/4, n/8, ……., 1)** pour conclure que x n'existe pas.

$$\frac{n}{2^{q-1}} = 1 \Rightarrow 2^{q-1} = n$$

$$\Rightarrow q - 1 = log_2(n) \Rightarrow q = Log_2(n) + 1$$

Au pire, l'algorithme devra s'exécuter q fois, par conséquent la complexité de ce dernier est $O(T) = O(C8 + C9 * q) = O(q) = O(Log_2(n) + 1) = O(Log_2(n))$ **(logarithmique)**

SOURCES DE CE COURS

- Frédéric Vivien, Algorithmique avancée, École Normale Supérieure de Lyon, 2002., pp. 93. Disponible sur http://perso.ens-lyon.fr/frederic.vivien/Enseignement/Algo-2001-2002/Cours.pdf
- Slim Msfar, Algorithmique et Complexité, 2012, pp 104. Disponible sur http://p835.phpnet.org/testremorque/upload/catalogue/coursalgorithmi.pdf
- Djamel-Eddine ZEGOUR, O-Notation, École nationale Supérieure d'Informatique, pp 33. Disponible sur http://www.zegour.netii.net/Site%20secondaire/Mcp/Cours%20ppt/1_o-notation.pdf
- S. Aroussi, Algorithmique et Structures de données, Université de Blida 1, 2018, Disponible sur https://sites.google.com/a/esi.dz/informatiqueblida